Double-click (or enter) to edit

```
# ============================================================
# EXPERIMENT NO. 6
# Predictive Analytics using Linear Regression
# and Model Performance Evaluation
# Dataset: California Housing Dataset
# ============================================================


# ============================================================
# STEP 1: IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)


# ============================================================
# STEP 2: UPLOAD DATASET
# ============================================================

print("Upload your California Housing CSV file:")

uploaded = files.upload()

# Automatically find uploaded CSV file
file_name = list(uploaded.keys())[0]

print("\nFile Uploaded Successfully:")
print(file_name)


# ============================================================
# STEP 3: LOAD DATASET
# ============================================================

df = pd.read_csv(file_name)

print("\nFirst 5 Rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())


# ============================================================
# STEP 4: BASIC DATA INFORMATION
# ============================================================

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())


# ============================================================
# STEP 5: REMOVE MISSING VALUES
# ============================================================

df = df.dropna()
```

```
print("\nDataset Shape After Removing Missing Values:")
print(df.shape)


# ============================================================
# STEP 6: CHECK DATASET COLUMNS
# ============================================================

print("\nAvailable Columns:")
print(df.columns.tolist())


# ============================================================
# STEP 7: SELECT FEATURES AND TARGET
# ============================================================

# California Housing datasets may use different target names.
# This code automatically detects common target names.

possible_targets = [
    "MedHouseVal",
    "median_house_value",
    "Price",
    "price",
    "target"
]

target_column = None

for col in possible_targets:
    if col in df.columns:
        target_column = col
        break

if target_column is None:
    print("\nTarget column was not automatically detected.")
    print("Please check the column names above.")

else:

    print("\nTarget Column:", target_column)

    # Independent variables
    X = df.drop(target_column, axis=1)

    # Keep only numerical columns
    X = X.select_dtypes(include=np.number)

    # Dependent variable
    y = df[target_column]

    print("\nIndependent Variables:")
    print(X.columns.tolist())

    print("\nDependent Variable:")
    print(target_column)


    # ============================================================
    # STEP 8: TRAIN-TEST SPLIT
    # 80% Training and 20% Testing
    # ============================================================

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42
    )

    print("\nTraining Data Shape:")
    print(X_train.shape)

    print("\nTesting Data Shape:")
    print(X_test.shape)


    # ============================================================
    # STEP 9: CREATE LINEAR REGRESSION MODEL
    # ============================================================

    model = LinearRegression()
```

```
# ========================================================
# STEP 10: TRAIN MODEL
# ========================================================

model.fit(X_train, y_train)

print("\nLinear Regression Model Trained Successfully!")


# ========================================================
# STEP 11: MAKE PREDICTIONS
# ========================================================

y_pred = model.predict(X_test)

print("\nFirst 10 Actual Values:")
print(y_test.head(10).values)

print("\nFirst 10 Predicted Values:")
print(y_pred[:10])


# ========================================================
# STEP 12: MODEL PERFORMANCE
# ========================================================

MAE = mean_absolute_error(y_test, y_pred)

MSE = mean_squared_error(y_test, y_pred)

RMSE = np.sqrt(MSE)

R2 = r2_score(y_test, y_pred)


print("\n================================================")
print("        MODEL PERFORMANCE METRICS")
print("================================================")

print("Mean Absolute Error (MAE):", round(MAE, 4))
print("Mean Squared Error (MSE):", round(MSE, 4))
print("Root Mean Squared Error (RMSE):", round(RMSE, 4))
print("R² Score:", round(R2, 4))


# ========================================================
# STEP 13: REGRESSION COEFFICIENTS
# ========================================================

coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print("\n================================================")
print("        REGRESSION COEFFICIENTS")
print("================================================")

print(coefficients)

print("\nIntercept:")
print(model.intercept_)


# ========================================================
# STEP 14: ACTUAL VS PREDICTED VALUES
# ========================================================

comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print("\n================================================")
print("       ACTUAL VS PREDICTED VALUES")
print("================================================")

print(comparison.head(10))


# ========================================================
# STEP 15: ACTUAL VS PREDICTED GRAPH
```

```
# ========================================================

plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.5
)

# Perfect prediction line
minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)

plt.title("Actual vs Predicted House Prices")
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")

plt.show()


# ========================================================
# STEP 16: REGRESSION LINE
# ========================================================

# Use first feature for simple visualization
first_feature = X.columns[0]

simple_X = df[[first_feature]]
simple_y = y

simple_model = LinearRegression()

simple_model.fit(
    simple_X,
    simple_y
)

x_line = np.linspace(
    simple_X[first_feature].min(),
    simple_X[first_feature].max(),
    100
).reshape(-1, 1)

y_line = simple_model.predict(x_line)

plt.figure(figsize=(8, 6))

plt.scatter(
    simple_X[first_feature],
    simple_y,
    alpha=0.3
)

plt.plot(
    x_line,
    y_line,
    linewidth=2
)

plt.title(
    "Regression Line: "
    + first_feature
    + " vs House Price"
)

plt.xlabel(first_feature)
plt.ylabel("House Price")

plt.show()


# ========================================================
# STEP 17: COEFFICIENT VISUALIZATION
# ========================================================

plt.figure(figsize=(10, 6))
```

```
    sns.barplot(
        data=coefficients,
        x="Coefficient",
        y="Feature"
    )

    plt.title("Regression Coefficients")
    plt.xlabel("Coefficient Value")
    plt.ylabel("Feature")

    plt.show()


    # ========================================================
    # STEP 18: RESIDUAL PLOT
    # ========================================================

    residuals = y_test - y_pred

    plt.figure(figsize=(8, 6))

    plt.scatter(
        y_pred,
        residuals,
        alpha=0.5
    )

    plt.axhline(
        y=0,
        linestyle="--"
    )

    plt.title("Residual Plot")
    plt.xlabel("Predicted Values")
    plt.ylabel("Residuals")

    plt.show()


    # ========================================================
    # STEP 19: FINAL RESULT
    # ========================================================

    print("\n================================================")
    print("             FINAL RESULT")
    print("================================================")

    print("MAE  =", round(MAE, 4))
    print("MSE  =", round(MSE, 4))
    print("RMSE =", round(RMSE, 4))
    print("R²   =", round(R2, 4))

    print("\nLinear Regression model completed successfully.")


# ============================================================
# QUESTIONS AND ANSWERS
# ============================================================


# Q1. What is Predictive Analytics, and how is it used in
# real-world applications?
#
# Answer:
# Predictive Analytics uses historical data, statistical methods
# and machine learning algorithms to predict future outcomes.
# It is used in sales forecasting, house price prediction,
# demand forecasting, risk analysis and customer behavior prediction.


# Q2. Explain the working principle of the Linear Regression
# algorithm.
#
# Answer:
# Linear Regression establishes a mathematical relationship between
# independent variables and a continuous dependent variable.
# It finds the best-fitting line by minimizing the difference between
# actual and predicted values.
#
# General equation:
#
# Y = b0 + b1X1 + b2X2 + ... + bnXn
#
```

```
# Where:
# Y = Predicted value
# b0 = Intercept
# b1, b2, ... = Regression coefficients
# X1, X2, ... = Independent variables


# Q3. Differentiate between dependent and independent variables
# with suitable examples.
#
# Answer:
# Independent variables are input variables used for prediction.
# The dependent variable is the output that we want to predict.
#
# Example:
# Independent variables = income, house age, rooms, population
# Dependent variable = house price


# Q4. Why is it necessary to split the dataset into training
# and testing sets?
#
# Answer:
# The training dataset is used to train the model.
# The testing dataset is used to evaluate the model on unseen data.
# It helps determine whether the model can generalize to new data.


# Q5. What is the significance of the R² Score in regression
# analysis?
#
# Answer:
# R² Score measures how much variation in the dependent variable
# is explained by the regression model.
#
# A value closer to 1 generally means that the model explains more
# variation in the target variable.


# Q6. Differentiate between MAE, MSE, and RMSE. Which metric is
# more sensitive to large prediction errors?
#
# Answer:
#
# MAE (Mean Absolute Error):
# It calculates the average absolute difference between actual
# and predicted values.
#
# MSE (Mean Squared Error):
# It calculates the average squared difference between actual
# and predicted values.
#
# RMSE (Root Mean Squared Error):
# It is the square root of MSE and is expressed in the same units
# as the target variable.
#
# MSE and RMSE are more sensitive to large prediction errors
# because the errors are squared.


# Q7. What assumptions should be satisfied before applying
# Linear Regression?
#
# Answer:
# Important assumptions are:
#
# 1. There should be a linear relationship between variables.
# 2. Observations should be independent.
# 3. Error variance should be approximately constant.
# 4. Residuals should be approximately normally distributed.
# 5. Independent variables should not have strong
#    multicollinearity.


# Q8. How can overfitting and underfitting affect the performance
# of a regression model?
#
# Answer:
# Overfitting occurs when a model learns the training data too
# closely and performs poorly on unseen data.
#
# Underfitting occurs when a model is too simple to capture the
# important patterns in the data.
```

```
#
# A good model should generalize well to unseen data.


# Q9. Mention any three real-world applications of Linear
# Regression in business or industry.
#
# Answer:
#
# 1. House price prediction
# 2. Sales forecasting
# 3. Demand or revenue prediction


# Q10. How can feature selection improve the accuracy and
# interpretability of a predictive model?
#
# Answer:
# Feature selection removes irrelevant or redundant variables.
# It can reduce noise, improve computational efficiency and make
# the model easier to understand.
#
# Selecting important features can also improve model performance
# when irrelevant variables are adding noise.


# ============================================================
# END OF EXPERIMENT NO. 6
# ============================================================
```